Flask Web Dashboard

This document describes the web dashboard's architecture, the Flask concepts applied, and how the backend communicates with the frontend.

Overview

The web dashboard is a server-side rendered Flask application. It provides administrators with analytics, sign-in logs, people management, and data export functionality through a standard web browser.

The dashboard code lives in the `dashboard/` directory: - `app.py` -- All routes, logic, and database queries. - `templates/` -- Jinja2 HTML templates. - `static/style.css` -- Stylesheet.

Flask Application Setup

The Flask app is created with standard configuration:

```
app = Flask(__name__)
app.secret_key = _resolve_secret_key()   # from DASHBOARD_SECRET_KEY env var, or a random fallback
```

The secret key is used for signing session cookies -- see 09-authentication-and-security.md for how it's sourced, plus CSRF protection (`flask-wtf`), login rate limiting (`flask-limiter`), and secure cookie settings, all configured at app-creation time alongside it. When running from a PyInstaller bundle, the template and static directories are resolved relative to the bundle's temporary extraction directory (`sys._MEIPASS`).

`init_db()` is called once at module import time (not just from the kiosk's `app.py`), so the dashboard can also be started completely standalone -- without the kiosk ever having run first -- and still have a working schema and a seeded login.

Routing

Flask uses decorators to map URL paths to Python functions. The dashboard defines these routes:

Route	Method	Access	Purpose
`/`	GET	any logged in	Redirect to dashboard (admin) or sign-in (staff)
`/login`	GET, POST	public	Login form and authentication (rate-limited)

`/logout`	GET	public	Clear session and redirect
`/signin`	GET, POST	any logged in	Web Sign In / Out page and submission
`/signin/search`	GET	any logged in	JSON: search members/staff by name for the sign-in page
`/signin/alerts`	GET	any logged in	JSON: active alerts for a set of member ids
`/status`	GET	any logged in	Read-only "who's here" view (staff's home page; admins can also reach it)
`/dashboard`	GET	admin	Analytics overview
`/logs`	GET	admin	Paginated sign-in log, with status/search/date filters
`/logs/sign/<int:sign_id>/edit`	GET, POST	admin	Edit an existing sign-in/out time
`/logs/sign/add`	GET, POST	admin	Add a missing sign-in/out time (`member_id`, `direction` query params)
`/audit`	GET	admin	Paginated audit log
`/people`	GET	admin	Member or staff listing
`/people/new`	GET, POST	admin	Create a new person
`/people/<int:person_id>`	GET, POST	admin	Edit an existing person
`/people/<int:person_id>/delete`	POST	admin	Delete a person
`/alerts`	GET, POST	admin	View/create sign-in alerts
`/alerts/<int:alert_id>/delete`	POST	admin	Delete an alert
`/exports`	GET	admin	Export page

`/export/staff`	GET	admin	Download staff Excel file -- all staff (`date_from`/`date_to` required) or one staff member (`person_id`, dates optional)
`/export/person`	GET	admin	Download one person's Excel file (`person_id` required; `date_from`/`date_to` optional query params)
`/people/<int:person_id>/signature.png`	GET	admin	Stream that person's most recently used signature as a PNG (404 if none on file)
`/dashboard-users`	GET	admin	List dashboard login accounts
`/dashboard-users/new`	POST	admin	Create a dashboard login (name + PIN + role)
`/dashboard-users/<int:user_id>/delete`	POST	admin	Delete a dashboard login (blocked for the last remaining admin)

Authentication

The dashboard authenticates against the `dashboard_users` table (name + hashed PIN), not a hardcoded credential -- see 09-authentication-and-security.md for the full design (hashing, roles, CSRF, rate limiting, secure cookies). In short:

1. User submits name and PIN via the login form (rate-limited to 5 attempts/minute).
2. The server looks up `database.check_dashboard_login(name, pin)`.
3. On success, the session stores `user_id`, `name`, and `role` (not just a boolean).
4. Two decorators guard routes: `login_required` (any authenticated user) and `admin_required` (must additionally have `role == "admin"`).

```
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not session.get("user_id"):
            return redirect(url_for("login"))
        return f(*args, **kwargs)
    return decorated
```

```
def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not session.get("user_id"):
            return redirect(url_for("login"))
        if session.get("role") != "admin":
            flash("Admin access required.", "error")
            return redirect(url_for("signin"))
        return f(*args, **kwargs)
    return decorated
```

The `@wraps(f)` decorator from `functools` preserves the original function's name and docstring, which Flask needs for `url_for()` to work correctly.

Template Rendering (Jinja2)

Flask uses Jinja2 as its template engine. Templates live in `dashboard/templates/` and use template inheritance:

Base Template (`base.html`)

Defines the page skeleton: HTML head, navigation bar, flash message area, and a content block. All other templates extend this:

```
{% extends "base.html" %}
{% block content %}
 <!-- page-specific content -->
{% endblock %}
```

Template Variables

Python data is passed to templates via `render_template()`:

```
return render_template(
    "dashboard.html",
    today_ins=today_ins,
    daily_labels=json.dumps(daily_labels),
    daily_values=json.dumps(daily_values),
    # ...
)
```

Lists and dictionaries intended for JavaScript (chart data) are serialised to JSON with `json.dumps()` in Python and then parsed in the template with Jinja2's `| safe` filter or `| tojson`.

Flash Messages

The dashboard uses Flask's `flash()` system for one-time status messages:

```
flash("Changes saved.", "success")
flash("First name and last name are required.", "error")
```

These are rendered in the base template after redirect, providing user feedback for form submissions.

Database Access

The dashboard has its own `get_conn()` function that connects to the same SQLite database as the kiosk. For operations that involve the kiosk's data models (creating/updating members, caregivers), it imports functions directly from the kiosk's `database.py` module:

```
from database import upsert_member, upsert_caregiver
```

This avoids duplicating business logic. The imports are done inside route functions (lazy imports) rather than at module level to avoid import order issues when both the kiosk and dashboard initialise.

Analytics Queries

The dashboard route performs several aggregate queries to compute statistics:

Currently Signed In

Uses a correlated subquery to find members whose most recent sign event today has direction "IN":

```
SELECT COUNT(*) FROM (
   SELECT s.member_id FROM sign_ins s
   WHERE DATE(s.timestamp) = ?
   AND s.direction = 'IN'
   AND s.id = (
        SELECT MAX(id) FROM sign_ins s2
        WHERE s2.member_id = s.member_id
        AND DATE(s2.timestamp) = ?
   )
)
```

This finds each member's latest sign event today (by maximum ID) and counts only those where the direction is IN.

Daily Totals

Groups sign events by date for the last 30 days. The Python code then fills in gaps (days with zero events) to produce a continuous series for the chart:

```
for i in range(30, -1, -1):
   d = (date.today() - timedelta(days=i)).isoformat()
   daily_labels.append(d[5:])
```

```
    daily_values.append(daily_map.get(d, 0))
```

Peak Hours

Uses SQLite's `strftime('%H', timestamp)` to extract the hour from each sign event and groups by hour across all time.

Weekly and Monthly Totals

Uses `strftime('%Y-W%W', timestamp)` and `strftime('%Y-%m', timestamp)` respectively to group sign events into week and month buckets.

Pagination

The logs, audit, and people pages implement server-side pagination:

```
page = max(1, int(request.args.get("page", 1)))
per_page = 50
offset = (page - 1) * per_page
total_pages = max(1, (total + per_page - 1) // per_page)
```

The total count is queried first, then the page's rows are fetched with `LIMIT ? OFFSET ?`. Page navigation links are generated in the template.

Filtering

The logs page supports filtering by name search and date range using query parameters, applied at the SQL level; the status filter (still signed in / signed out) is applied afterwards in Python by `_pair_sign_in_visits`, since "signed in" vs. "signed out" is a property of a paired *visit*, not a single `sign_ins` row. The SQL-level conditions are dynamically built:

```
where_parts = []
params = []
if date_from:
    where_parts.append("DATE(s.timestamp) >= ?")
    params.append(date_from)
if date_to:
    where_parts.append("DATE(s.timestamp) <= ?")
    params.append(date_to)
if search:
    where_parts.append("(m.first_name || ' ' || m.last_name) LIKE ?")
    params.append(f"%{search}%")
where_clause = ("WHERE " + " AND ".join(where_parts)) if where_parts else ""
```

This approach safely constructs SQL WHERE clauses while still using parameterised queries to prevent injection. After the matching rows are paired into visits, `status=OPEN`/`status=CLOSED` filters the resulting list down to open (no sign-out yet) or closed visits.

File Downloads

The export routes generate Excel files in memory using `io.BytesIO()` and send them to the browser with `send_file()`:

```
buf = io.BytesIO()
generate_staff_export(signs, date_from, date_to, buf)
buf.seek(0)
return send_file(buf, as_attachment=True, download_name=filename,
                 mimetype="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
```

The `as_attachment=True` flag tells the browser to download the file rather than displaying it. The MIME type specifies the `.xlsx` format.

Charts (Client-Side)

The dashboard page uses Chart.js (loaded from a CDN in the template) to render interactive charts. Chart data is injected into the HTML as JSON:

```
<script>
 const dailyLabels = {{ daily_labels | safe }};
 const dailyValues = {{ daily_values | safe }};
 new Chart(ctx, { type: 'bar', data: { labels: dailyLabels, datasets: [...] } });
</script>
```

The `| safe` filter tells Jinja2 not to escape the JSON string, since it was already serialised safely in Python.

Form Handling

People creation and editing use standard HTML forms with POST method. Flask's `request.form` dictionary provides access to submitted values:

```
data = {
   "first_name": request.form.get("first_name", "").strip(),
   "last_name":  request.form.get("last_name", "").strip(),
   # ...
}
```

Multi-value form fields (like day checkboxes) use `request.form.getlist()` and are joined into comma-separated strings for storage.

After a successful save, the pattern is Post/Redirect/Get (PRG): the route processes the form, then redirects to the edit page. This prevents duplicate submissions if the user refreshes the page.

Members vs. Staff (`person_type`)

Members and staff are both rows in the same `members` table, distinguished by a `person_type` column (`"member"` or `"staff"`; `NULL` is treated as `"member"` for rows created before the column existed). A single template, `person_form.html`, renders both the "new"/"edit member" screen and the "new"/"edit staff" screen -- it switches between the full member form (child details, caregiver, attendance days, activities) and the short staff form (name, role, PIN) based on a `person_type` variable passed in from the route.

Because both screens share one `<form>`, `person_type` has to be threaded through explicitly as a hidden field:

```
<input type="hidden" name="person_type" value="{{ person_type }}">
```

On the **create** path (`/people/new`), this hidden field is the *only* source of truth for which type is being created -- `app.py`'s `person_new()` reads it with `request.form.get("person_type", "member")`. If that hidden input is ever removed or renamed, every new "staff" submission silently defaults to `"member"` regardless of which tab (Members or Staff) the admin was on, and the post-create redirect then renders the member form instead of the staff form for the record just created. This exact bug shipped once; see `10-testing.md` for the regression tests that now guard against it.

On the **edit** path (`/people/<id>`), `person_type` is *not* read from the form at all -- `person_edit()` looks it up from the existing database row (`person.get("person_type")`) and reuses that value, since a person's type isn't meant to change after creation. The hidden field is rendered on the edit screen too (for template consistency) but has no effect on the POST handler.

Web Sign In / Out Page

`/signin` reimplements the kiosk's sign-in/out flow (`kiosk_signin/screens/signature_screen.py`) as a web page, reusing the exact same data-layer functions rather than duplicating the logic: `get_allowed_direction()`, `record_sign()`, `get_alerts_for_members()`, and `email_notifier.send_notification()`. The route, `signin.html`, and `static/signin.js` work together:

1. `signin.js` debounces keystrokes in the search box and calls `GET /signin/search?type=member|staff&q=...`, which returns JSON `{"results": [{"id": ..., "name": ...}]}` from `search_members()`/`search_staff()`.
2. Selecting a result adds them to an in-page "selected" list (chips), and triggers `GET /signin/alerts?member_ids=1,2,3` -- if any alerts come back, a modal shows them before the direction/notes/signature section is revealed, mirroring the kiosk's pre-submit alert popup.
3. On submit, the browser POSTs to `/signin` with repeated `member_ids` fields, `direction`, `notes`, and `signature_data`.

Signature Format Bridging

The kiosk's `SignatureCanvas` stores a signature as a Python list of `(x1, y1, x2, y2)` line-segment tuples, serialised with `str(strokes).encode()` -- and `export_util.render_signature()` decodes it back with `ast.literal_eval()`. For the web signature pad to produce signatures that exports and the `/people/<id>/signature.png` route can read without any changes, `/signin`'s POST handler converts the browser's stroke data into that exact byte format server-side:

```
def _parse_signature(raw: str):
    strokes = json.loads(raw)                       # browser sends JSON: [[x1,y1,x2,y2], ...]
    ...                                              # validated: must be a list of 4-number lists
    return str([tuple(s) for s in clean]).encode()  # same format as SignatureCanvas.export_bytes()
```

The browser never builds the Python-literal string itself -- it only ever sends plain JSON, which is what browsers can serialise losslessly. Converting and validating server-side means malformed or oversized input (capped at 20,000 segments) is rejected with a flash message rather than silently producing a blob that would later fail `ast.literal_eval()` at export time. A signature is optional, matching the kiosk (which no longer requires one either) -- an empty `signature_data` field just results in `signature=None` being passed to `record_sign()`.

Direction Validation and Partial Success

Just like the kiosk, submitting a batch of people can partially succeed: each selected person is checked individually against `get_allowed_direction()`, so if one person in a group is already signed in and the rest aren't, only the eligible ones are recorded and the flash message reports how many were skipped -- nobody's submission is rejected outright just because one person in the group wasn't eligible.